

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Exp1 = pro1.c

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <stdlib.h>

int main()

{

    char command[100];

    printf("Enter a Linux command: ");

    scanf("%s", command);

    pid_t pid = fork();

    if (pid < 0)

    {

        perror("fork failed");

        return 1;

    }

    else if (pid == 0)
```

```
{  
    printf("Child Process\n");  
    printf("Child PID: %d\n", getpid());  
    printf("Parent PID: %d\n", getppid());  
  
    execlp(command, command, NULL);  
  
    perror("exec failed");  
    exit(1);  
}  
else  
{  
    printf("Parent Process\n");  
    printf("Parent PID: %d\n", getpid());  
    printf("Child PID: %d\n", pid);  
  
    wait(NULL);  
  
    printf("Child process completed.\n");  
}  
  
return 0;  
}
```

Output :

```

skjuneeth@LAPTOP-D8SJ2A02:~$ nano pro1.c
skjuneeth@LAPTOP-D8SJ2A02:~$ gcc pro1.c -o pro1
skjuneeth@LAPTOP-D8SJ2A02:~$ ./pro1
Enter a Linux command: ls
Parent Process
Parent PID: 628
Child PID: 629
Child Process
Child PID: 629
Parent PID: 628
OSPP
Team18_OSSP      display.c  filecall  input.txt  operating.c  p3.c  p6.c  p9.c  pro_con.c  zombie_fixed
allow.c          exp1.c.save filecall.c llo.        p1           p4.c  p7.c  parent    read        zombie_fixed.c
command_executor exp2.c      fork       main        p1.c         p4.c  p7.c  parent.c  read.c
command_executor exp3.c      fork.c     main.c      p2           p5.c  p8.c  pro1      week2s.c
display          exp3.c      hello      myshell     p2.c         p5.c  p8.c  pro1.c   zombie
Child process completed.
skjuneeth@LAPTOP-D8SJ2A02:~$ |

```

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

Exp2 = pro2.c

```

#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Before fork():\n");

    printf("Process PID : %d\n", getpid());

    printf("Parent PPID : %d\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");

        return 1;
    }
}

```

```
}
```

```
else if (pid == 0)
```

```
{
```

```
    // Child process
```

```
    printf("\n--- CHILD PROCESS ---\n");
```

```
    printf("Child PID   : %d\n", getpid());
```

```
    printf("Child PPID  : %d\n", getppid());
```

```
    printf("Child State : Running\n");
```

```
    sleep(5);
```

```
    printf("Child State : Terminating\n");
```

```
    exit(0);
```

```
}
```

```
else
```

```
{
```

```
    // Parent process
```

```
    printf("\n--- PARENT PROCESS ---\n");
```

```
    printf("Parent PID   : %d\n", getpid());
```

```
    printf("Parent PPID  : %d\n", getppid());
```

```
    printf("Child PID   : %d\n", pid);
```

```
    printf("Parent State : Running\n");
```

```
    printf("\nParent waiting for child...\n");
```

```
    wait(NULL);
```

```
    printf("Child terminated.\n");
```

```
    printf("Parent State : Running\n");
```

```
    printf("Parent State : Terminated after completion\n");
```

```
}
```

```
return 0;
```

```
}
```

Output :

```
skjuneeth@LAPTOP-D8SJ2A02:~$ nano pro2.c
skjuneeth@LAPTOP-D8SJ2A02:~$ gcc pro2.c -o pro2
skjuneeth@LAPTOP-D8SJ2A02:~$ ./pro2
Before fork():
Process PID   : 669
Parent PPID   : 330

--- PARENT PROCESS ---
Parent PID    : 669
Parent PPID   : 330
Child PID     : 670
Parent State  : Running

Parent waiting for child...

--- CHILD PROCESS ---
Child PID     : 670
Child PPID    : 669
Child State   : Running
Child State   : Terminating
Child terminated.
Parent State  : Running
Parent State  : Terminated after completion
skjuneeth@LAPTOP-D8SJ2A02:~$ |
```

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Exp3 = pro3.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
int main()
{
    pid_t pid;

    printf("Before fork()\n");
    printf("Process PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(10);

        printf("Child process terminating...\n");
    }
    else
    {

```

```

    printf("\nParent Process\n");

    printf("PID : %d\n", getpid());

    printf("PPID : %d\n", getppid());


    sleep(5);


    printf("Parent process terminating...\n");
}

return 0;
}

```

Output:

```

skjuneeth@LAPTOP-D8SJ2A02:~$ nano pro3.c
skjuneeth@LAPTOP-D8SJ2A02:~$ gcc pro3.c -o pro3
skjuneeth@LAPTOP-D8SJ2A02:~$ ./pro3
Before fork()
Process PID: 707

Parent Process
PID : 707
PPID : 330

Child Process
PID : 708
PPID : 707
Parent process terminating...
skjuneeth@LAPTOP-D8SJ2A02:~$ |

```

Exp 4 = pro4.c

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

exp4.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
    pid_t pid;
```

```
    for (i = 0; i < 3; i++)
```

```
    {
```

```
        pid = fork();
```

```
        if (pid < 0)
```

```
        {
```

```
            perror("fork failed");
```

```
            exit(1);
```

```
        }
```

```
        if (pid == 0)
```

```
        {
```

```
            printf("Child %d: PID = %d, PPID = %d\n",
```

```
                i + 1, getpid(), getppid());
```

```
            sleep(i + 1);
```

```
            printf("Child %d completed.\n", i + 1);
```

```
            exit(0);
```

```
        }
```



```
}
```

```
for (i = 0; i < 3; i++)
```

```
{
```

```
    wait(NULL);
```

```
    printf("Parent collected a child.\n");
```

```
}
```

```
printf("All children completed.\n");
```

```
return 0;
```

```
}
```

```
skjuneeth@LAPTOP-D8SJ2A02:~$ nano pro4.c
skjuneeth@LAPTOP-D8SJ2A02:~$ gcc pro4.c -o pro4
skjuneeth@LAPTOP-D8SJ2A02:~$ ./pro4
Child 1: PID = 743, PPID = 742
Child 2: PID = 744, PPID = 742
Child 3: PID = 745, PPID = 742
Child 1 completed.
Parent collected a child.
Child 2 completed.
Parent collected a child.
Child 3 completed.
Parent collected a child.
All children completed.
skjuneeth@LAPTOP-D8SJ2A02:~$ |
```